



FLORIDA STATE UNIVERSITY
DEPARTMENT OF SCIENTIFIC COMPUTING

WRITTEN PRELIMINARY EXAMINATION
SUMMER 2024

Question - 8

Submitted By:
Mani Tyagi

Student ID :
200973108
mt22t

May 23, 2024

Contents

1	Parallel Programming [Dr. Zavala]	2
1.1	OpenMP	2
1.2	MPI	4

1 Parallel Programming[Dr. Zavala]

1.1 OpenMP

The function to be integrated is

$$h(x) = e^x$$

Simpson integration can be used to approximate this integral. The formula is given by:

$$I \approx \frac{h}{3} \left[f(x_0) + 4 \sum_{i=1}^{n/2} f(x_{2i-1}) + 2 \sum_{i=1}^{n/2-1} f(x_{2i}) + f(x_n) \right]$$

Results for n=1000:

Threads	Integral	Error	Time (microseconds)
1	1.71828	6.66134×10^{-15}	35
2	1.71828	3.77476×10^{-15}	105
4	1.71828	2.26485×10^{-15}	88
6	1.71828	6.43929×10^{-15}	122
8	1.71828	5.55112×10^{-15}	89
10	1.71828	4.88498×10^{-15}	76
16	1.71828	4.44089×10^{-16}	115

Table 1: Integration Results with $n = 10^3$

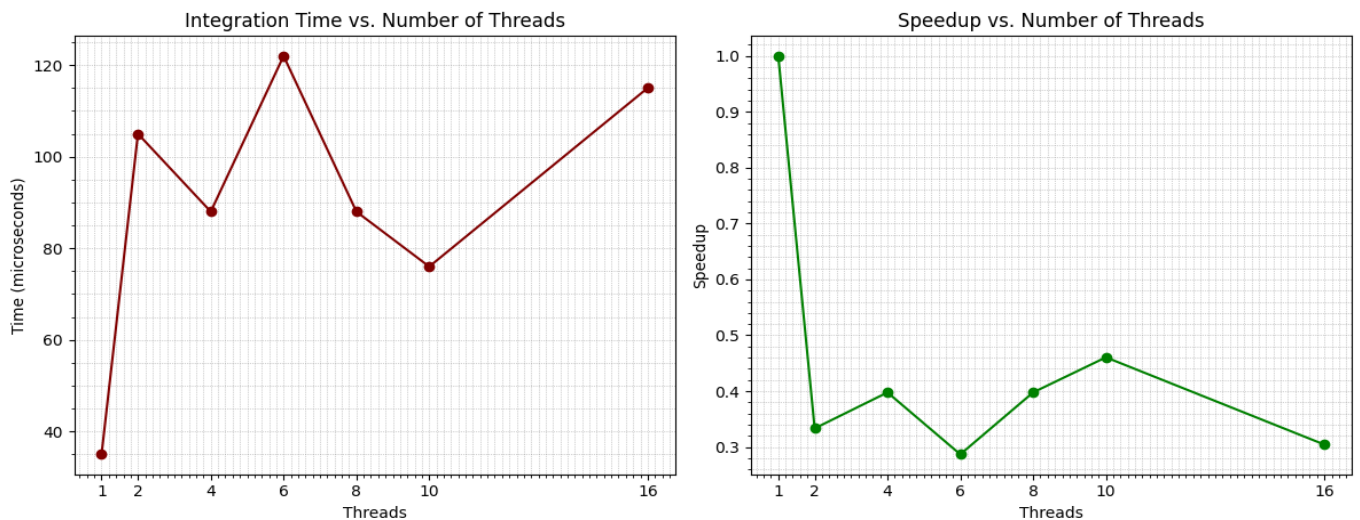


Figure 1: Performance of the OpenMp code with $n = 10^3$ sections

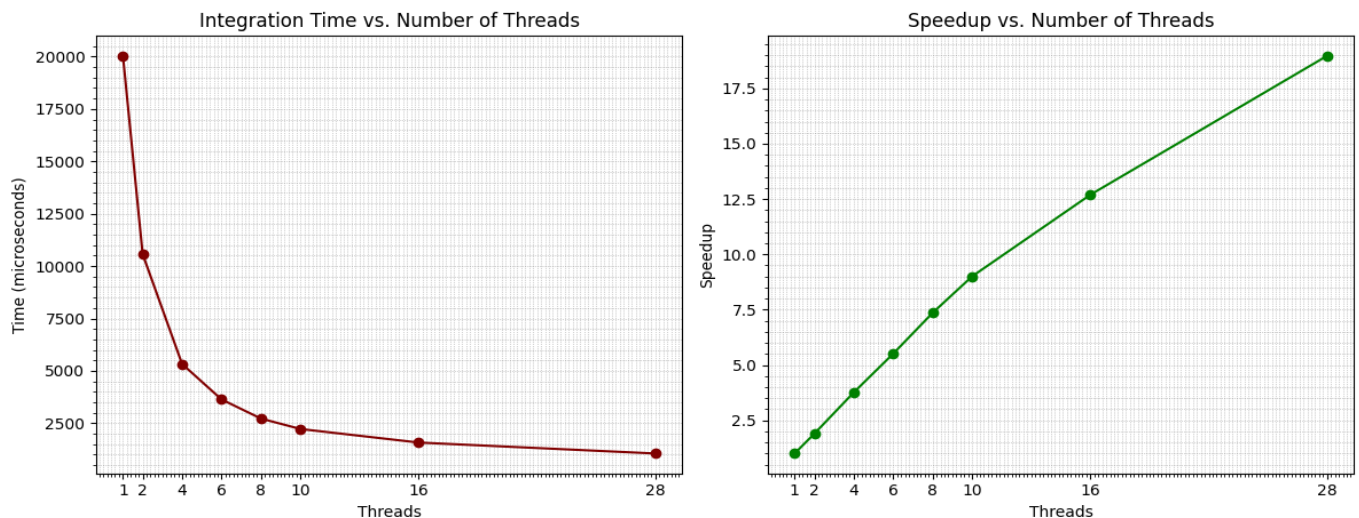
Discussion and Explanation: As clear from Table-1 and Fig-1, the speedup of the code is very poor. With increase in number of threads, the code shows inconsistent behaviour. The reason for this poor performance is [Parallelization Overhead](#) which is occurring because of very small computation workload per thread. For $n = 1000$, the overhead of creating and managing multiple threads is outweighing the benefits of parallelization, leading to suboptimal performance.

The code gives very good speedup for larger values of n . For demonstration, I have done the calculations with $n = 1000000$ sections.

Results for $n=1000000$:

As shown in Table-2 and Fig-2, the speedup obtained for $n=1000000$ is very good (almost linear). In this case, the benefits of parallelization are outweighing the overhead of creating and managing multiple threads.

Threads	Integral	Error	Time (microseconds)
1	1.71828	6.66134×10^{-15}	20035
2	1.71828	3.77476×10^{-15}	10570
4	1.71828	2.26485×10^{-15}	5327
6	1.71828	6.43929×10^{-15}	3613
8	1.71828	5.55112×10^{-15}	2723
10	1.71828	4.88498×10^{-15}	2225
16	1.71828	4.44089×10^{-16}	1578
28	1.71828	4.44089×10^{-16}	1056

Table 2: Integration Results with $n = 10^6$ Figure 2: Performance of the OpenMp code with $n = 10^6$ sections**NOTES:**

- I have used classroom machines to run OpenMP codes. There are 28 threads in total on those machines. A simple code `max_threads.cpp` is used to get the number of threads.
- The openMP implementation is present in `8_openmp.cpp`. This code uses a function file `simpson.h` which I already had from Assignment 12 (FALL23) Scientific Programming course.
- **Compilation Commands used:**

```
-----
[mt22t@class106 Q8-200973108]$ g++ -fopenmp 8_openmp.cpp -o openmp
[mt22t@class106 Q8-200973108]$ ./openmp
-----
```

1.2 MPI

I got a lot of bugs in my initial code and I used chatGpt for debugging them. I have attached the conversation at the end.

Code Implementation Details

1. **MPI Initialization** : MPI is initialized using `MPI_Init` function, and the rank and size of the MPI processes are obtained using `MPI_Comm_rank` and `MPI_Comm_size` functions, respectively.
2. **Division of Interval** The interval $[0, 1]$ is evenly divided into $2N_p$ subintervals, where N_p is the number of processes.
3. **Local Integration** Each process computes its local integral using Simpson's rule for the assigned subintervals.
4. **Ring-like Communication** Processes exchange function values in a ring-like fashion using `MPI_Send` and `MPI_Recv` functions.
5. **Gathering Results** The local results are gathered on process 0 using `MPI_Gather` function.

Process	Area (sq. units)
0	0.104908
1	0.115941
2	0.128135
3	0.141611
4	0.156504
5	0.172964
6	0.191155
7	0.211259
8	0.233477
9	0.258032
Full approx.	1.71399

Table 3: Integration Results using MPI.

NOTES:

- I have used classroom machines to run MPI codes.
- The MPI implementation is present in `8_MPI.cpp`. This code is based on [Moreno Marzolla's](#) code.
- **Compilation Commands used:**

```
-----
[mt22t@class106 Q8-200973108]$ mpic++ 8_MPI.cpp -o MPI
[mt22t@class106 Q8-200973108]$ mpirun -np 10 MPI
-----
```

References

- Scientific Programming Assignment 12, Dr. Erlebacher
- High Performance computing Lecture Notes on OpenMP, Dr. Wang
- ChatGpt conversation: <https://chatgpt.com/share/afee595c-0ed8-4257-9279-eb0cfbf55252>
- <https://www.moreno.marzolla.name/teaching/HPC/handouts/mpi-ring.html>